Device Trees

by Kowshic Shankar R

Introduction

- Embedded systems, with their diverse components and architectures, require an efficient way to describe hardware in a standardized, reusable format.
- The Device Tree (DT) provides a flexible structure to address this need, defining hardware details in a way that the operating system and bootloader can interpret.

Discoverable vs. Non-Discoverable Hardware

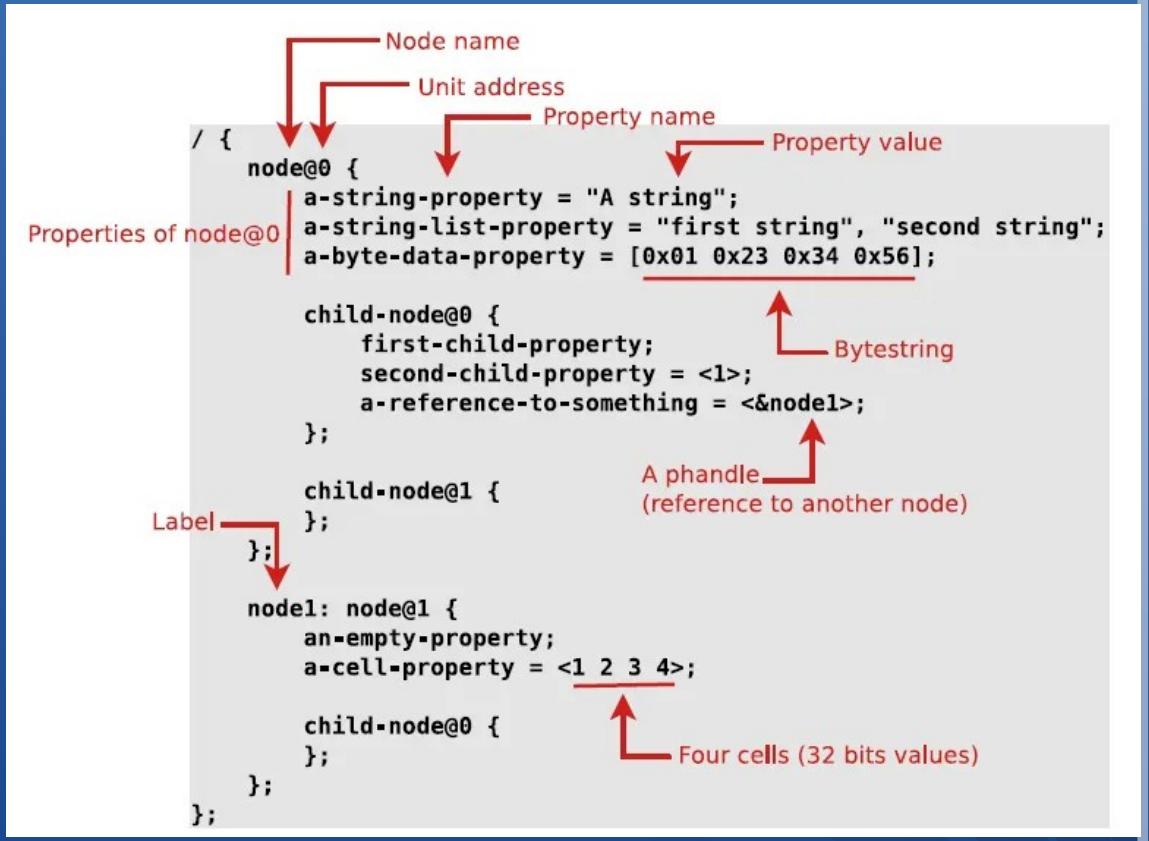
- Discoverable Hardware
  - Some buses, like PCI(e) and USB, support device discoverability, meaning they can enumerate and identify connected devices at runtime.
  - Devices on these buses can communicate details like vendor ID, product ID, and device class to the system.
- Non-Discoverable Hardware
  - Buses like I2C, SPI, and 1-Wire lack discoverability. Here, the system must know what devices are connected and how they’re configured.
  - Device Tree is especially useful in embedded systems that rely heavily on these types of buses.
- Methods of handling Non-Discoverable hardwares are : Directly in the OS, ACPI Tables, Decvice Trees.

Principles of Device Tree

Device Tree consists of a tree data structure describing hardware, typically written in Device Tree Source (.dts) files, then compiled into Device Tree Blob (.dtb) format by the Device Tree Compiler (dtc).

Syntax of Device Tree

- Nodes represent devices.
- Properties define characteristics of these devices.
- Each node may refer to other nodes using a special concept known as the phandle.



Key Device Tree Properties

- reg: Defines base addresses and register sizes for memory-mapped devices.
- interrupts: Specifies the interrupt line used by a device and the interrupt controller it connects to.
- clocks: References the clock(s) used by the device.
- dmas: Lists DMA controllers and channels.
- status: okay indicates the device is active and ready for use.
- pinctrl-*: Specifies pin-muxing configurations.

Device Tree Inheritance

- Device Trees support a modular design where properties are inherited from other files.
- For example, .dtsi files contain SoC-level details, while .dts files define board-specific configurations.
- This structure reduces redundancy and simplifies the process of defining common elements across similar boards.

Example

https://gitlab.utc.fr/mi11/linux/linux-raspberrypi/-/blob/9af1cc301e4dffb830025207a54d0bc63bec16c7/arch/arm/boot/dts/bcm2711-rpi-4-b.dts

Reference

- https://medium.com/@hasancansert/mastering-device-trees-a-guide-to-hardware-integration-in-linux-3e1516a75e04